



di.unito.it

DIPARTIMENTO
DI INFORMATICA

TLN

rappresentazioni vettoriali di
significato: word2vec

Daniele Radicioni

outline

- word embeddings
 - Word2vec
 - training and classification task
 - further sets of embeddings (GloVe, fasttext)
 - similarity and analogy with pretrained embeddings
- LABS
 - Document Similarity and Retrieval using the VSM
 - Word Clustering Based on Word2Vec Representation

credits

- Jurafsky, D. (2000). Speech & language processing.
Pearson Education India.
 - Chapter 6: 6.8
- books and online resources on Word2vec, mentioned in the following

from long and sparse vectors to
short and dense vectors
or word embeddings

from *documents* as vectors...

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
fool	36	58	1	4

to *words* as vectors

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

- what if we consider row vectors?
 - for documents, we said that similar documents tend to have similar (words, and thus) vectors
 - same applies to words: similar words tend to have similar vectors, as they tend to occur in similar documents: we will focus on this in next future, when introducing *word embeddings*

which features

- hand-crafted features must be designed and implemented by the machine learning developer
 - **should be syntax employed** to perform text categorization? how can **negation** be handled (clearly plain bag of words methods are not enough)?
- **most words in any language tend to be very infrequent.**
 - if one ranks the words in a language in descending order of their frequency then the frequency of the word at rank i is $1/i$ times the frequency of the most frequent word. Zipfian distribution, power law

semantic similarity vs. lexicalization

- if the training data for a review classification dataset contain the word *great* but not the word *fantastic*, a learning algorithm trained on this data will not be able to properly handle reviews containing the latter word
 - even though there is a clear semantic similarity between the two
- however, the context surrounding both terms will be similar: if their meaning is similar, we can expect them to be involved in similar utterances

first intuition

- Harris (1954): words that occur in similar contexts tend to have similar meanings; Firth (1957): a word is characterized by the company it keeps
 - reworded, we could say that *linguistic items with similar distributions have similar meanings.*

.

first intuition

- Harris (1954): words that occur in similar contexts tend to have similar meanings; Firth (1957): a word is characterized by the company it keeps
 - reworded, we could say that *linguistic items with similar distributions have similar meanings.*
- each word in a given vocabulary is provided with a vector representing the context in which the word occurs.
 - according to the distributional hypothesis, these vectors should capture the semantic meaning of words, and, thus, words that are similar should have similar vectors.

vectorial representation

- for each word w in the vocabulary, its vector counts the co-occurrence with other words in its surrounding context, where this context is defined as a window of size $[-c, +c]$ words around all instances of w in text
 - negative values indicate number of words to the left of w , and positive values to indicate number of words to its right.
 - let us suppose we are given three sentences containing the word bagel, and to set $c = 3$ (c size usually ranges between 10 and 20) such as
 - "A **bagel** and cream cheese"
 - "also known as **bagel** with cream cheese"
 - "American cuisine The **bagel** is typically sliced "

co-occurrence vector

- the **aggregated word counts** for the considered sentences are in the table
 - "A **bagel** and cream cheese"
 - "also known as **bagel** with cream cheese"
 - "American cuisine The **bagel** is typically sliced "
- co-occurrence vector indeed grasps meaningful contextual information: bagel is most strongly associated with *cream* and *cheese*, but also with other relevant context words such as *cuisine* and *sliced*.

A	1
also	1
American	1
and	1
as	1
cheese	2
cream	2
cuisine	1
is	1
known	1
sliced	1
The	1
typically	1
with	1

co-occurrence vector

- to collect co-occurring elements one may drop words deemed to contain minimal semantic meaning such as determiners, pronouns, and prepositions.
 - also, words may be normalized in some form, e.g., through lemmatization
- co-occurrence vectors are typically generated from large document collections such as Wikipedia, and have size M , which is the size of the whole vocabulary (that is, all words in the document collection)
 - such vectors are sparse, that is they mostly contain 0 values.

Word2vec

Jurafsky, 6.8

embeddings

- in VSM we have introduced how to represent a word as a sparse, long vector with dimensions corresponding to words in the vocabulary or documents in a collection
- we now introduce embeddings: **short** and **dense vectors**
 - embeddings are **short**, with number of dimensions d ranging from 50-1000, rather than the much larger vocabulary size $|V|$ or number of documents D we've seen
 - embeddings are **dense**: instead of vector entries being sparse, the values will be real-valued numbers that can be negative

embeddings

- dense vectors work better in every NLP task than sparse vectors.
 - while we don't completely understand all the reasons for this, we have some intuitions.
 - representing words as 300-dimensional dense vectors requires our classifiers to learn far fewer weights than if we represented words as 50,000-dimensional vectors, and **the smaller parameter space possibly helps with generalization and avoiding overfitting.**

Word2vec skip-gram

- the goal of this algorithm is to learn numerical representations that capture that distributional hypothesis
 - while the original hypothesis stated that "a word is characterized by the company it keeps", i.e., a word is defined by its context, word2vec's training objective predicts the context in which a given word is likely to occur, i.e., the context is defined by the word.
 - we focus on the skip-gram algorithm

skip-gram algorithm

- the method to compute embeddings is **skip-gram** with negative sampling (SGNS)
 - the skip-gram algorithm is one of two algorithms in a software package called word2vec, and so sometimes the algorithm is loosely referred to as word2vec
- intuition: instead of counting how often each word w occurs near, say, *apricot*, **we train a classifier on a binary prediction task: "Is word w likely to show up near *apricot*?"**
 - we don't actually care about this prediction task; instead we take the learned classifier weights as the word embeddings.

skip-gram algorithm

- the idea is that we use running text as implicitly supervised training data for such a classifier; a word c that occurs near the target word *apricot* acts as gold 'correct answer' to the question "is word c likely to show up near *apricot*?"
 - this method, often called self-supervision, avoids the need for any sort of hand-labeled supervision signal
- Word2vec simplifies the task, by making it binary classification instead of word prediction
 - word2vec simplifies the architecture, by training a (logistic regression) classifier instead of a multi-layer neural network with hidden layers that demand more sophisticated training algorithms

skip-gram in a nutshell

- the main steps of the algorithm are as follows:
 - treat the target word and a neighboring context word as positive examples
 - randomly sample other words (outside the neighboring context!) from the lexicon to get negative samples
 - use logistic regression to train a classifier to distinguish those two cases
 - use the learned weights as the embeddings.

collecting the context

- let us start from the sentence "Thou shalt not make a machine in the likeness of a human mind"
 - assume we have a context of two words before/after the target word
 - the sentence is sampled through a sliding window, whereby each word is, in turn, the target word

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word

collecting the context

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word



Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a

- four samples are collected (one for each context word)

collecting the context

- few steps later

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

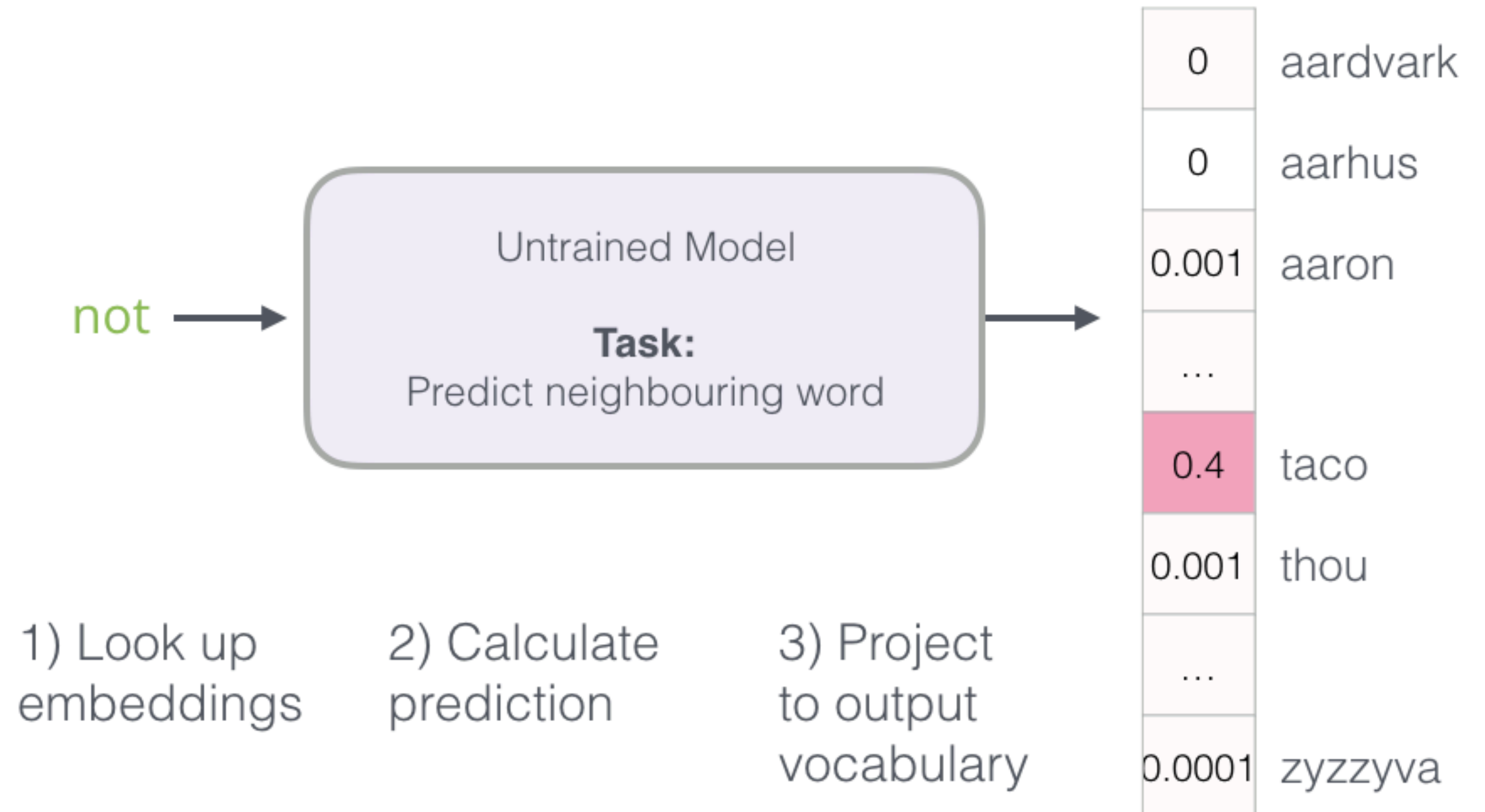
thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

neighboring word prediction

- the untrained model is used to predict neighboring words
- the **output is a prediction vector** (where a probability is assigned to each word in the vocabulary)



neighboring word prediction

- assessment of the model's output

Actual
Target

0
0
0
...
0
1
...
0

-

Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	
0.4	taco
0.001	thou
...	
0.0001	zyzzyva

=

Error

0
0
-0.001
...
-0.4
0.999
...
-0.0001

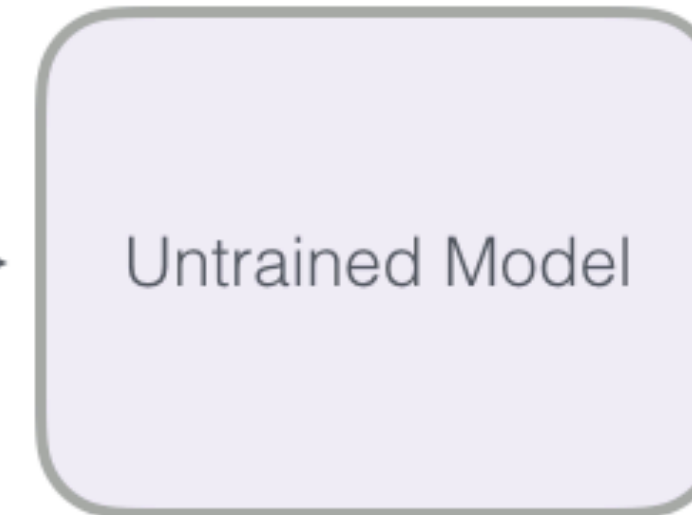
learning step

- the error vector is then used to update the model

Actual
Target

0
0
0
...
0
1
...
0

not



Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	...
0.4	taco
0.001	thou
...	...
0.0001	zyzzyva

Error

0
0
-0.001
...
-0.4
0.999
...
-0.0001

Update
Model
Parameters

learning step

- the learning step is computationally expensive, since it must be repeated for all training examples
- the task is then turned into a simpler one: instead of predicting the neighboring word (given an input word), the model might be just requested to (given an input word and an output word) output a score stating whether these are neighbors or not
 - from a neural network to a logistic regression model, which is simpler and faster to compute

negative sampling

- 0 stands for "not neighbors", 1 for "neighbors"

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

negative samples

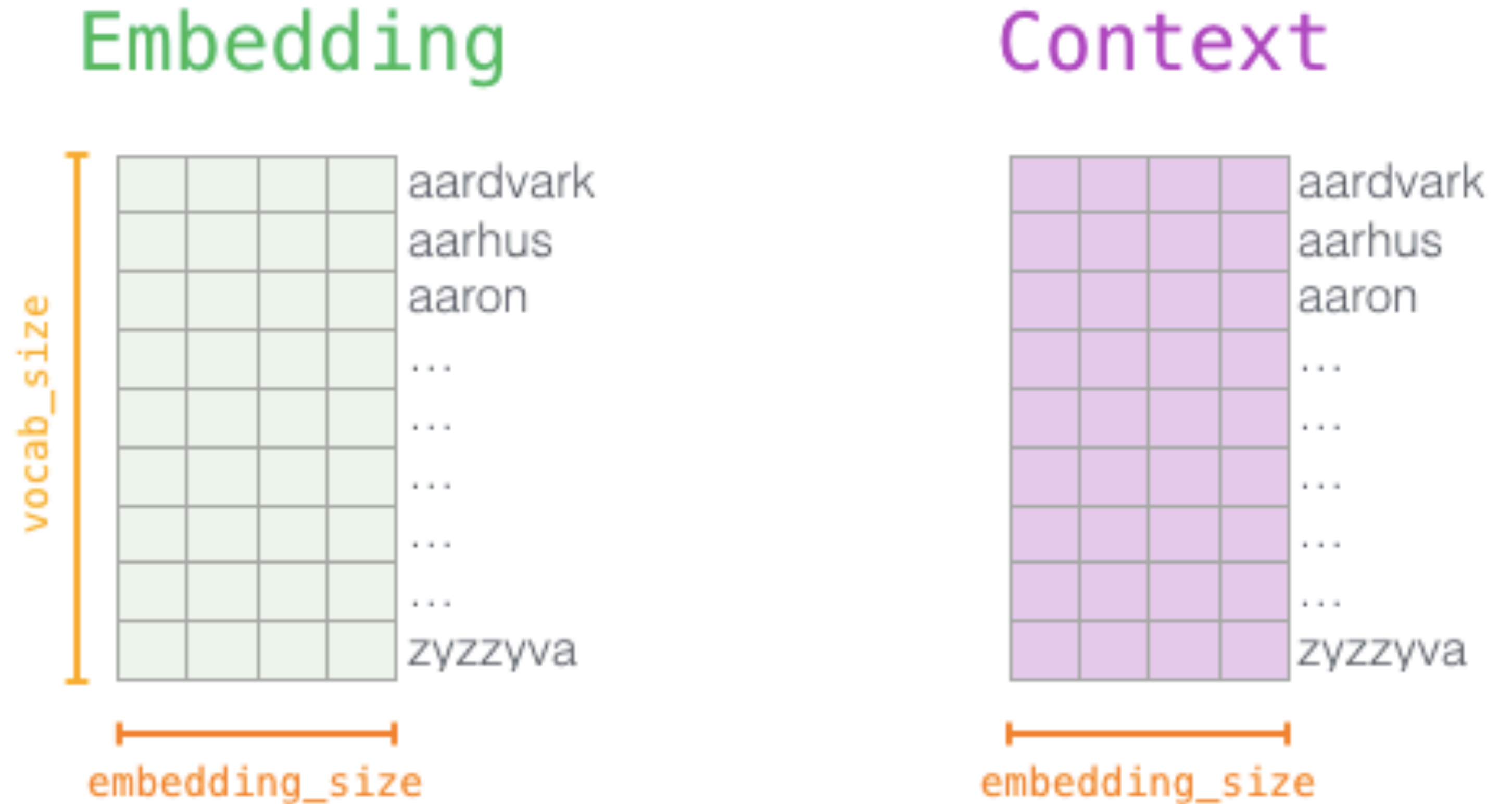
input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1

 Negative examples

- providing the model with discriminative features involves considering even negative examples
- add **negative samples**: samples of words that are not neighbors
 - 0 stands for "not neighbors", 1 for "neighbors"

training

- two matrices – an Embedding matrix and a Context matrix– are created (e.g., with 10k words vocabulary size, and 300 dimensions)



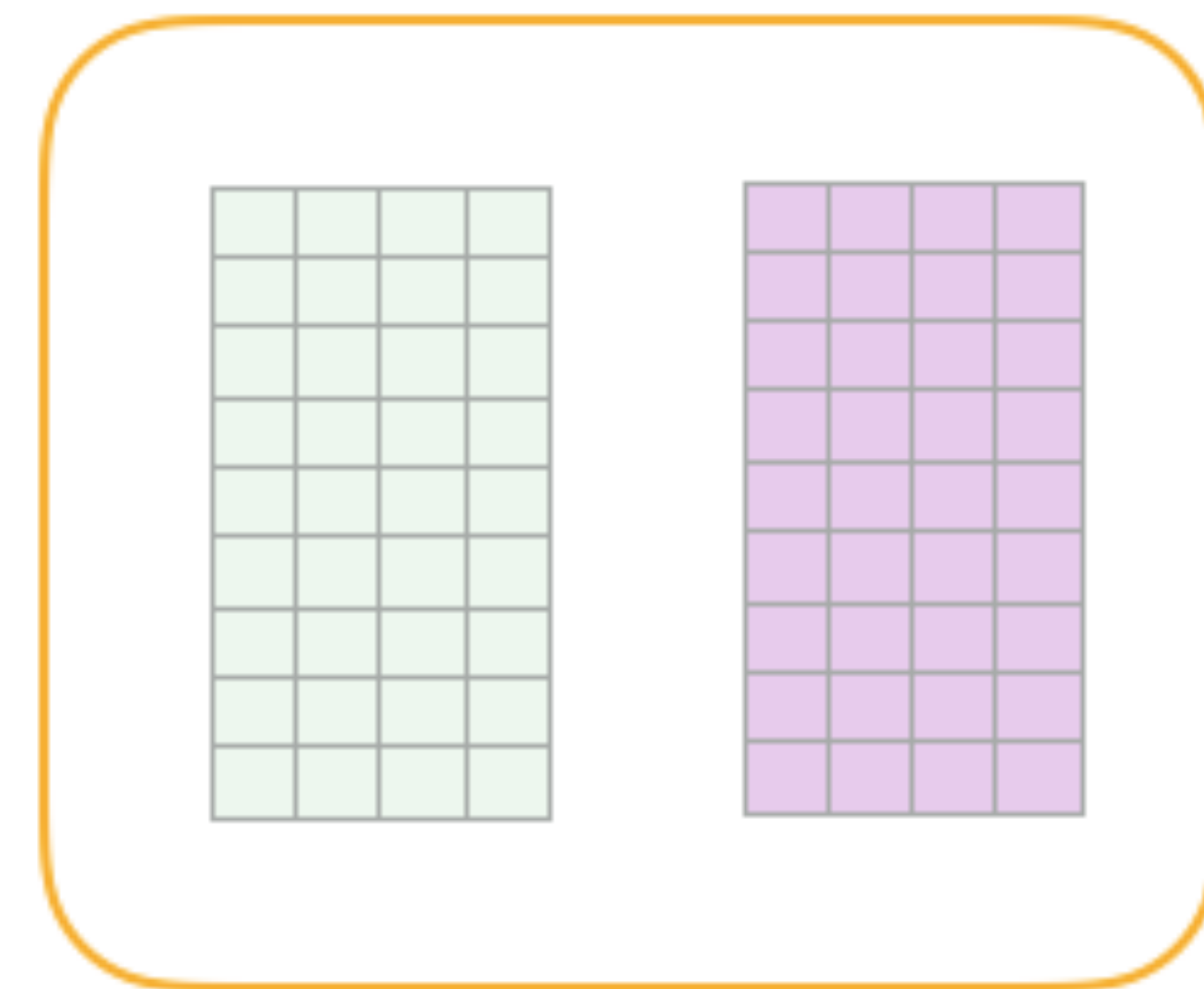
training

- such matrices are initialized with random values
- in each training step, we take one positive example and its associated negative examples

dataset

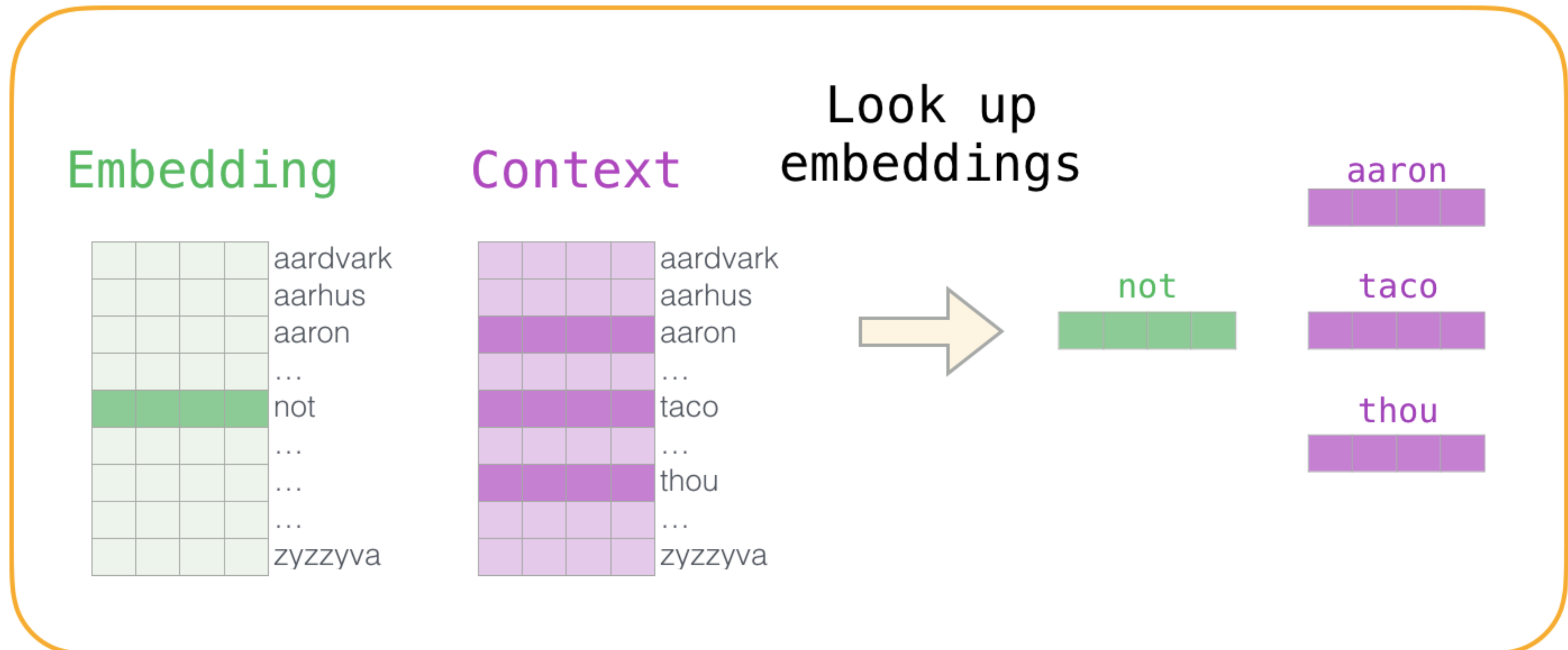
input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

model



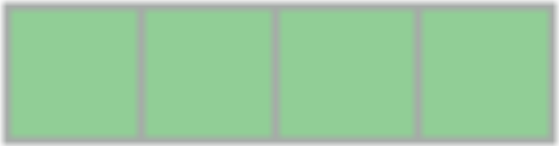
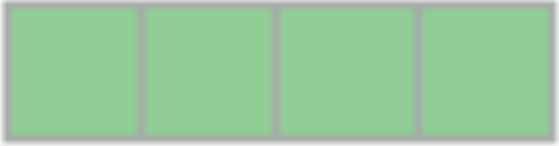
training

- given the input word **not** and output/context words: **thou** (the actual neighbor), **aaron**, and **taco** (the negative examples).
 - for the input word, we look up its embedding from Embedding matrix.
 - for the context words, we take the embeddings from the Context matrix



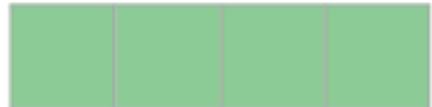
training

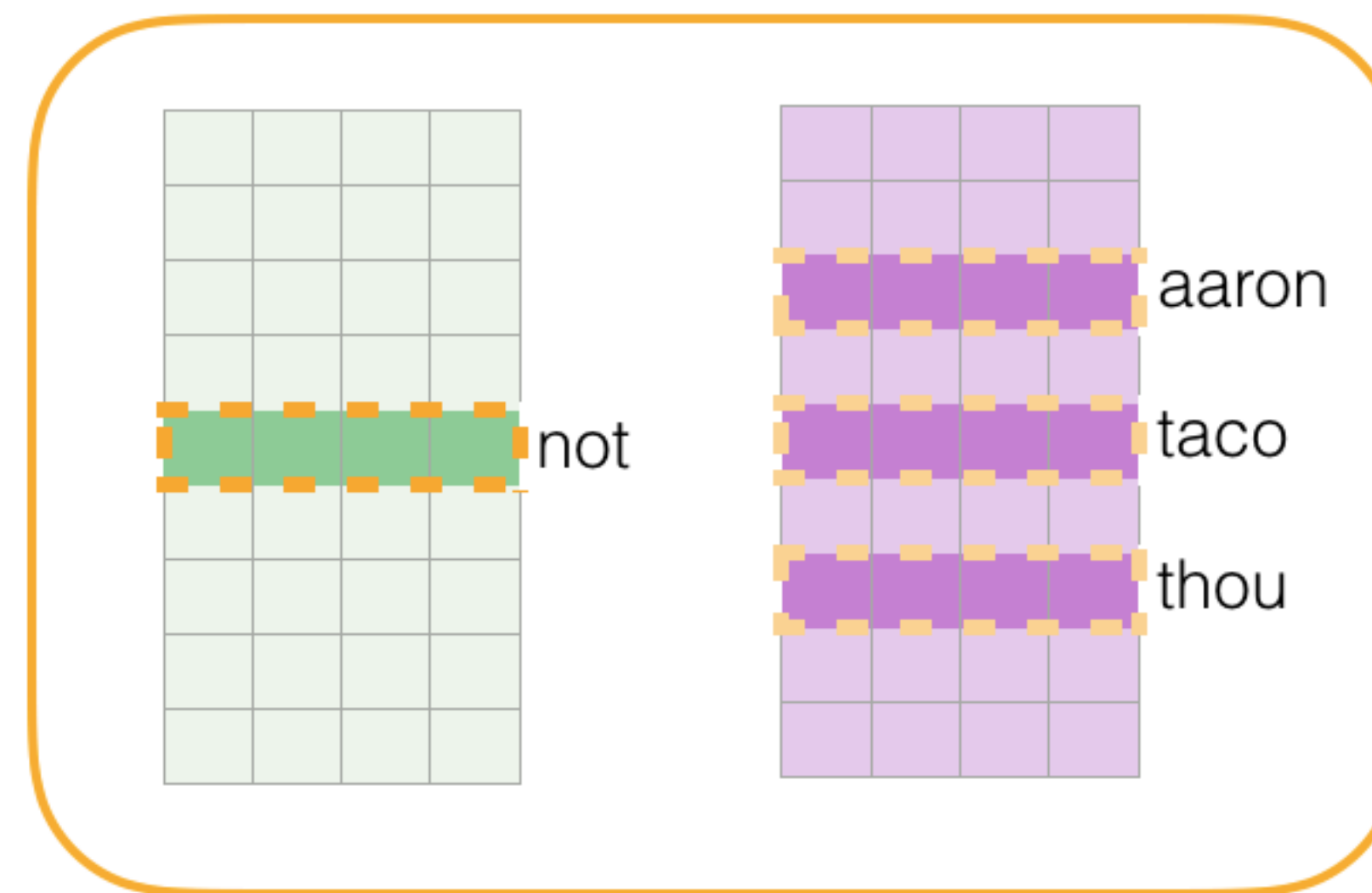
- we take the dot product of the input embedding with each of the context embeddings
- we then turn these scores into positive values ranging in $[0,1]$.
 - this is done through the logistic function.

input word	output word	target	input • output	sigmoid()
not 	thou 	1	0.2	0.55
not 	aaron 	0	-1.11	0.25
not 	taco 	0	0.74	0.68

error estimation

- computed by subtracting the sigmoid scores from the target labels
- parameters are updated on this basis

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68



Update
Model
Parameters

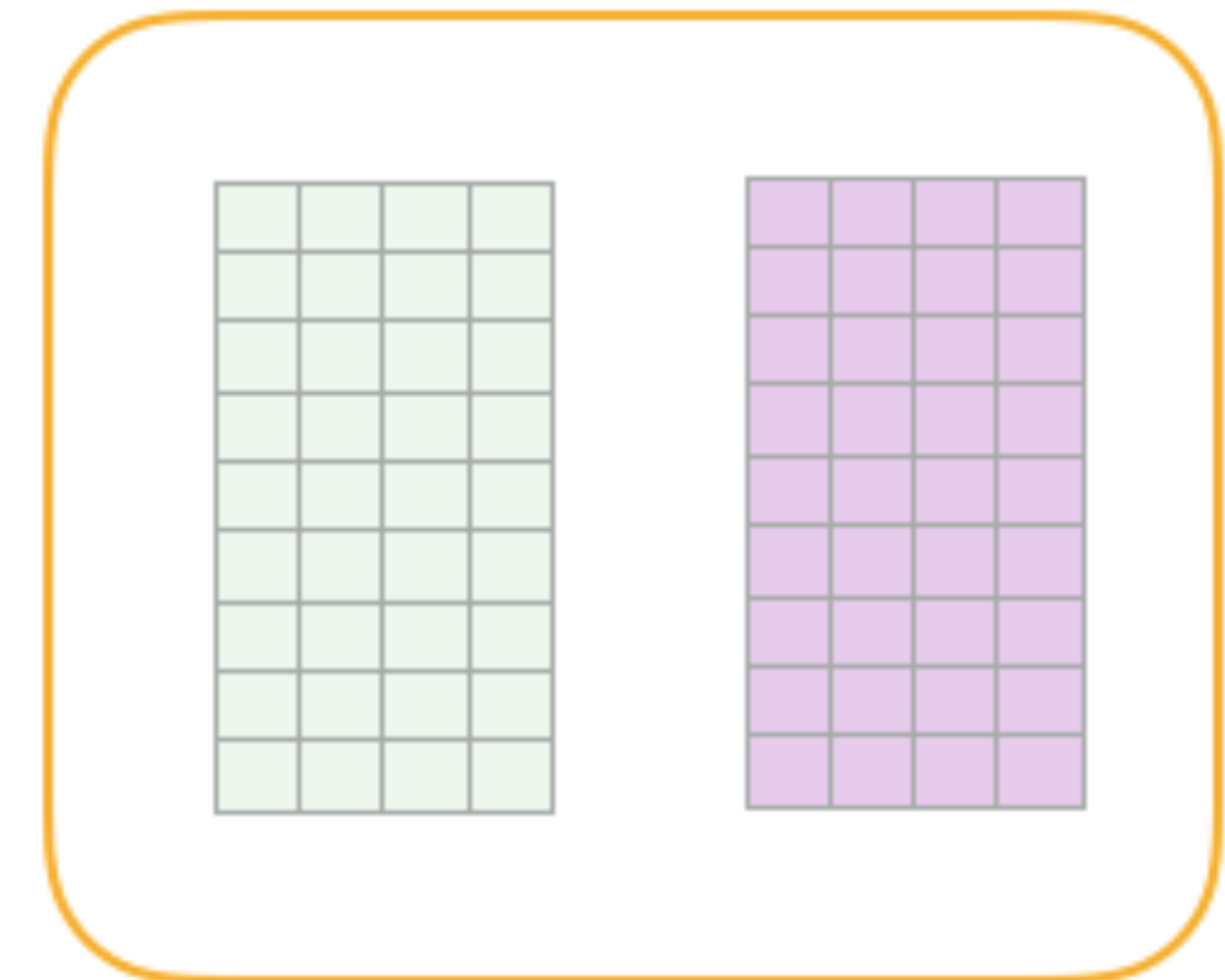
loop over examples

- cycle through the entire dataset for a number of times

dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

model



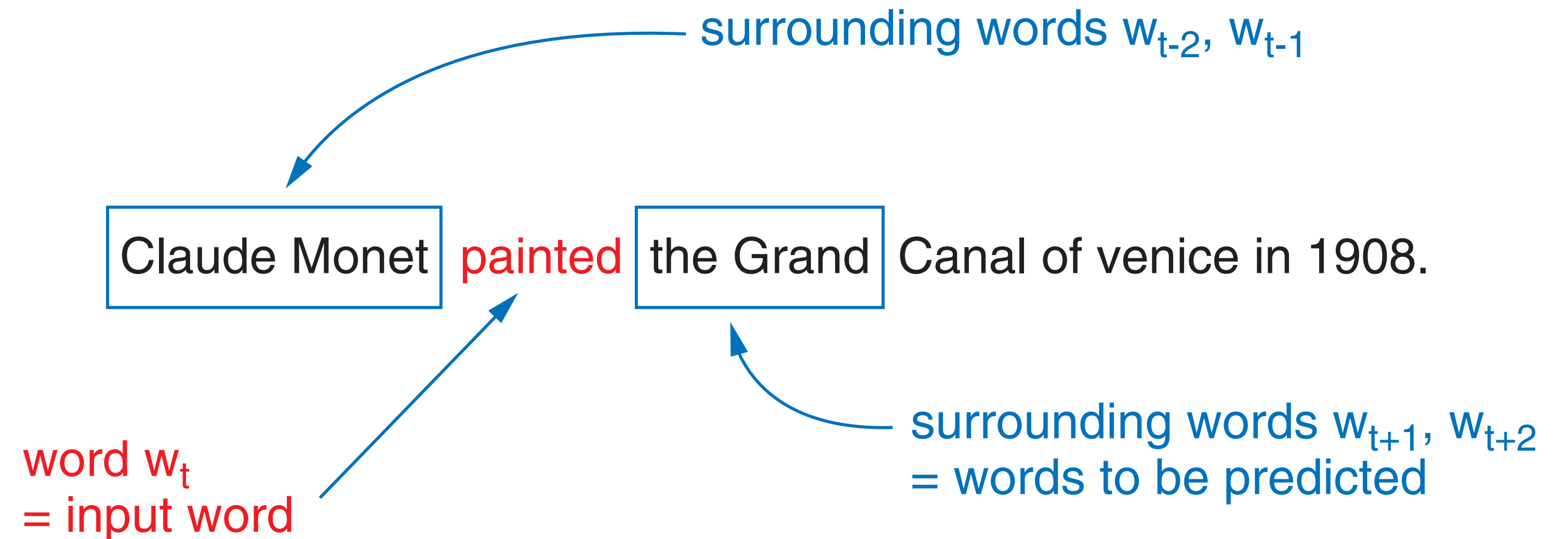
skipgram

further architectural details

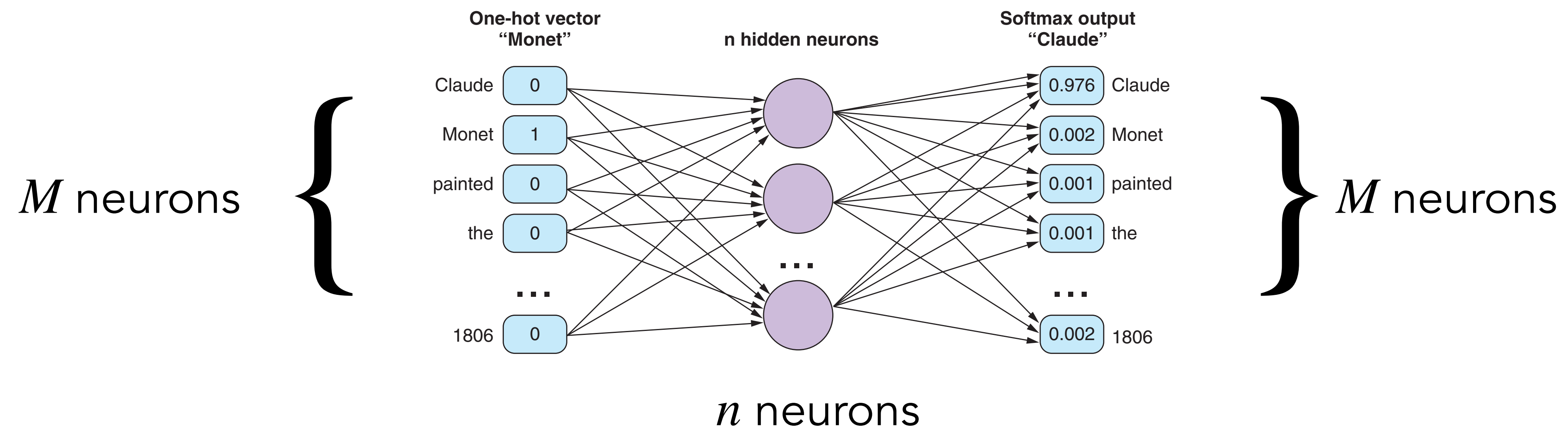
credits

- next slides contain material coming from
 - Hobson Lane, Cole Howard, Hannes Max Hapke, *Natural Language Processing in Action. Understanding, analyzing, and generating text with Python*, Manning Publications, 2019, chap 6: *Reasoning with word vectors (Word2vec)*

skip-gram



- Skip-grams are n -grams that contain gaps because we skip over intervening tokens.
 - in the example, we are predicting "Claude" from the input token "painted," and we skip over the token "Monet."
- in the skip-gram training approach, we are trying to predict the surrounding window of words based on an input word.



- the network consists of two layers of weights, where the hidden layer consists of n neurons; n is the number of vector dimensions used to represent a word. both the input and output layers contain M neurons, where M is the number of words in the model's vocabulary.
- the output layer activation function is a softmax

one-hot vectors

- words are represented as one-hot vectors before they are presented to the network
- the softmax activation of the output layer nodes (one for each token in the vocabulary) calculates the probability of an output word being found as a surrounding word of the input word.
 - the output vector of word probabilities can then be converted into a one-hot vector where the word with the highest probability will be converted to 1, and all remaining terms will be set to 0.
 - this simplifies the loss calculation

one-hot vectors

- thanks to the one-hot vector conversion of text tokens, each row in the weight matrix represents each word from the vocabulary for your corpus. after the training, semantically similar words will have similar vectors, because they were trained to predict similar surrounding words.
- after the training is complete and we decide not to train your word model any further, the output layer of the network can be ignored. only the weights of the inputs to the hidden layer are used as the embeddings.
- or in other words: the weight matrix is our word embedding.

retrieving word vectors

One-hot vector
in vocabulary
of six words

0	1	0	0	0	0
---	---	---	---	---	---

Three neuron
weight matrix

.03	.92	.66
.06	.32	.61
.14	.62	.43
.24	.99	.62
.12	.02	.44
.32	.23	.55

The dot product calculation

$$(0 \cdot .03) + (1 \cdot .06) + (0 \cdot .14) + (0 \cdot .24) + (0 \cdot .12) + (0 \cdot .32)$$

$$(0 \cdot .92) + (1 \cdot .32) + (0 \cdot .62) + (0 \cdot .99) + (0 \cdot .02) + (0 \cdot .23)$$

$$(0 \cdot .66) + (1 \cdot .61) + (0 \cdot .43) + (0 \cdot .62) + (0 \cdot .44) + (0 \cdot .55)$$

=

.06
.32
.61

Resulting 3-D word vector

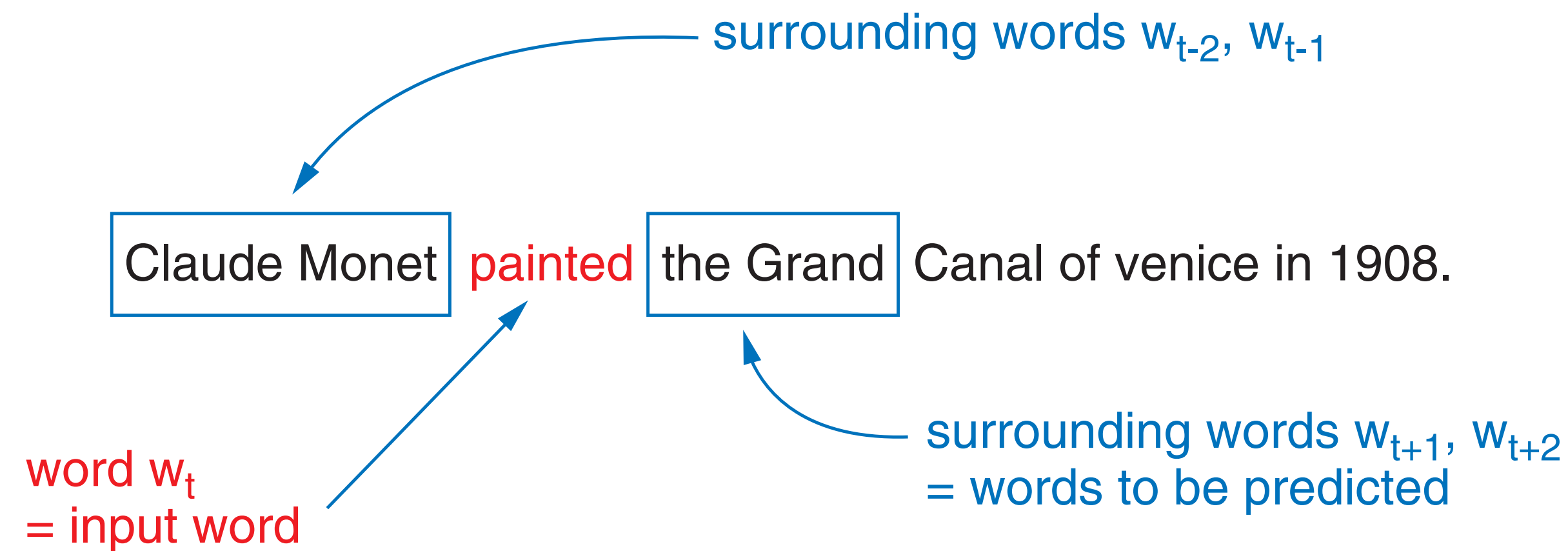
- The weights of a hidden layer in a neural network are often represented as a matrix: one column per input neuron, one row per output neuron. This allows the weight matrix to be multiplied by the column vector of inputs coming from the previous layer to generate a column vector of outputs going to the next layer

continuous bag-of-words, CBOW

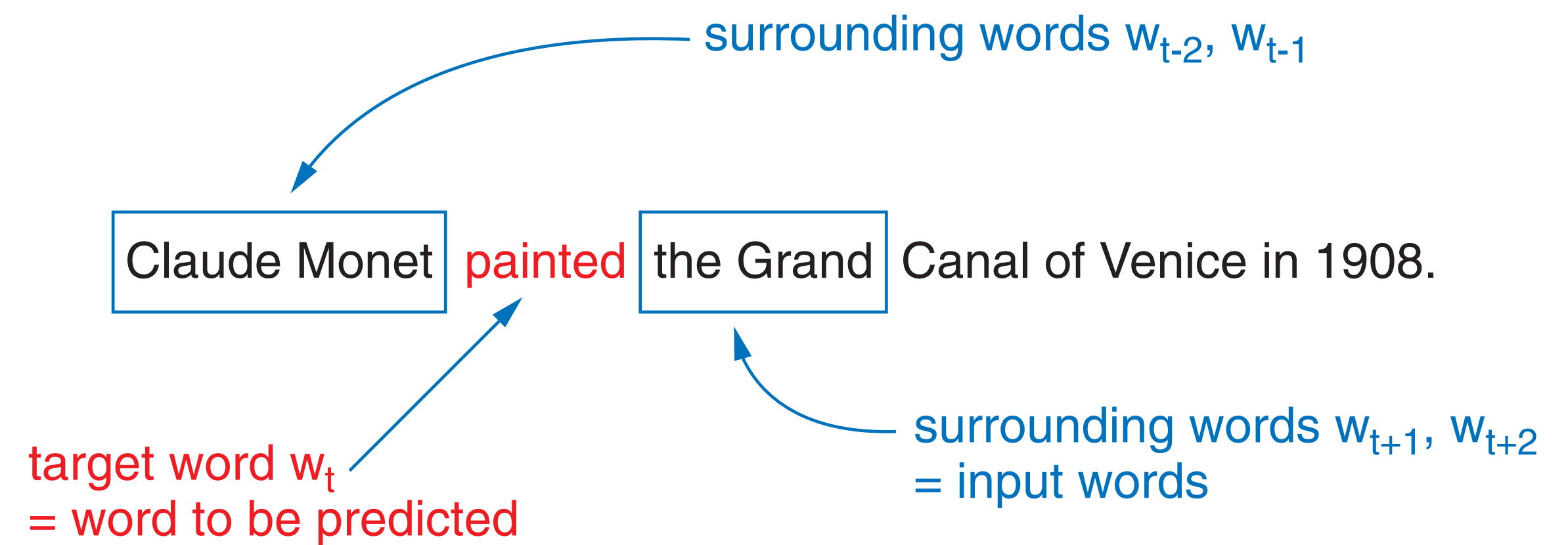
continuous bag-of-words

- how does a bag-of-words differ from a continuous bag of words (CBOW)?
 - to establish the relationships between words in a sentence you slide a rolling window across the sentence to select the surrounding words for the target word.
 - all words within the sliding window are considered to be the content of the continuous bag of words for the target word at the middle of that window.

skip-gram



CBoW



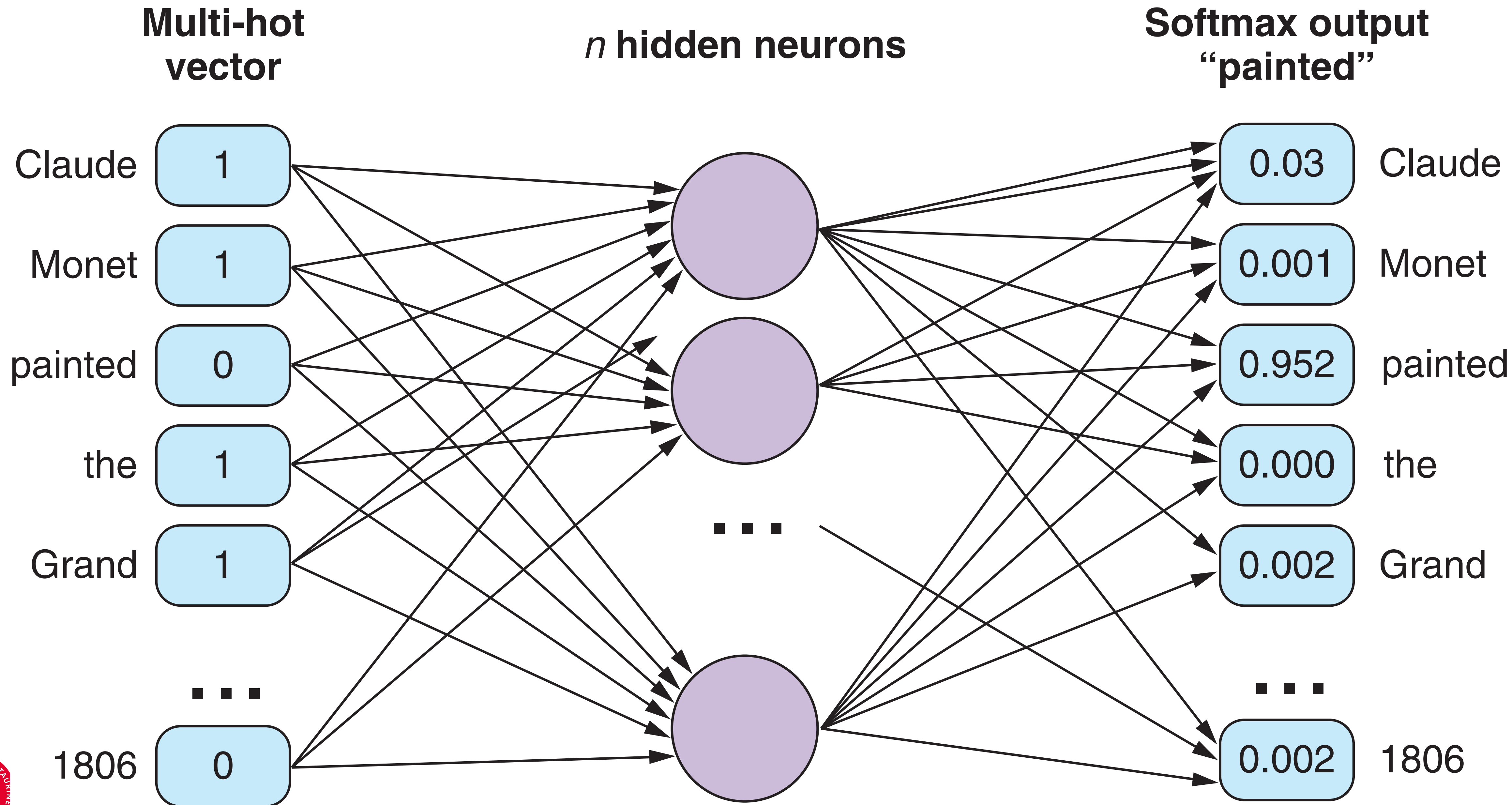
- in the continuous bag-of-words approach, we are trying to predict the center word based on the surrounding words
 - instead of creating pairs of input and output tokens, we create a multi-hot vector of all surrounding terms as an input vector.
 - the multi-hot input vector is the sum of all one-hot vectors of the surrounding tokens to the center, target token.

multi-hot vectors

- the multi-hot vector is the sum of the one-hot vectors of the surrounding words' training pairs

$$w_{t-2} + w_{t-1} + w_{t+1} + w_{t+2}$$

- the training pairs with the multi-hot vector are built as the input and the target word w_t as the output.
 - during the training, the output is derived from the softmax of the output node with the highest probability



classifier

the classification task

- imagine a sentence like the following, with a target word *apricot*, and assume we are using a **window of ± 2 context words**:

... lemon, a [tablespoon of apricot jam, a] pinch ...
 c1 c2 w c3 c4

- the goal is to train a classifier that takes in input a tuple (w, c) of a target word w paired with a candidate context word c , for example $(apricot, jam)$, or $(apricot, aardvark)$.
 - in the end the classifier should **return the probability that c is a real context word** (*true for jam, false for aardvark*): $P(+ | w, c)$
- the probability that word c is not a real context word for w is just 1 minus the previous one: $P(- | w, c) = 1 - P(+ | w, c)$

the classification task

- the skip-gram algorithm employs **embeddings similarity**:
a word is likely to occur near the target if its embedding is similar to the target embedding
 - two **vectors are similar if they have a high dot product**:
 $\text{similarity}(w, c) \approx c \cdot w$
 - the dot product $c \cdot w$ is not a probability, it is just a number ranging from $-\infty$ to ∞ (since the elements in word2vec embeddings can be negative, the dot product can be negative)

the classification task

- to turn the dot product into a probability, the logistic function $\sigma(x)$

is used: $\sigma(x) = \frac{1}{1 + \exp(-x)}$

- the probability that word c is a real context word for target word w

is modeled as: $P(+ | w, c) = \sigma(c \cdot x) = \frac{1}{1 + \exp(-c \cdot x)}$

- the sigmoid function returns a number between 0 and 1, but to make it a probability we also need the total probability of the two possible events (c is a context word, and c is not a context word) to sum to 1.
- we thus estimate the probability that word c is not a real context word

for w as: $P(- | w, c) = 1 - P(+ | w, c) = \sigma(-c \cdot x) = \frac{1}{1 + \exp(c \cdot x)}$

the classification task

$$P(+ | w, c) = \sigma(c \cdot x) = \frac{1}{1 + \exp(-c \cdot x)}$$

- the equation above gives the probability for one word: Skip-gram makes the **simplifying assumption that all context words are independent**, allowing us to just multiply their

probabilities:
$$P(+ | w, c_{1:L}) = \prod_{i=1}^L \sigma(-c_i \cdot w)$$

- $$\log P(+ | w, c_{1:L}) = \sum_{i=1}^L \log \sigma(-c_i \cdot w)$$

details

Word2vec skip-gram

- for each input word w_i in the vocabulary, the algorithm identifies the context windows of size $[-c, +c]$ around all instances of w_i in some large text
- then, all the context (or output) words that appear in these windows are added to the pool of words that should be predicted given w_i
- the training process then maximizes the prediction probability for each word w_j in the context of w_i . the **theoretical cost function** is

$$C = - \sum_{i=1}^M \sum_{w_j \in \text{ctx}(w_i)} \log \left(p(w_j | w_i) \right), \text{ where } M \text{ is the vocabulary size and}$$

$\text{ctx}(w_i)$ is the context of w_i

cost function

- the training process then maximizes the prediction probability for each word w_j in the context of w_i . the theoretical cost function is

$$C = - \sum_{i=1}^M \sum_{w_j \in \text{ctx}(w_i)} \log \left(p(w_j | w_i) \right), \text{ where}$$

- $\text{ctx}(w_i)$ is the context of w_i , and
- the probability $p(w_j | w_i)$ is computed using the input vector for w_i , the output

vector w_j and the softmax function: $p(w_i | w_j) = \frac{e^{\mathbf{v}_{w_j}^o \cdot \mathbf{v}_{w_i}^i}}{\sum_{k=1}^M e^{\mathbf{v}_{w_k}^o \cdot \mathbf{v}_{w_i}^i}}$, where $\mathbf{v}^i$

indicates an input vector and $\mathbf{v}^o$ a context (or output) vector, and the denominator in the fraction iterates over all the words in the vocabulary of size M in order to normalize the resulting probability

cost function

- $C = - \sum_{i=1}^M \sum_{w_j \in \text{ctx}(w_i)} \log(p(w_j | w_i)),$
 - minimizing C has the effect of maximizing the probabilities $p(w_j | w_i)$ due to the negative sign in the above equation
 - maximizing $p(w_j | w_i)$ brings the output vectors of context words ($\mathbf{v}_{w_j}^o$) and the input vector for w_i ($\mathbf{v}_{w_i}^i$) closer, since this maximizes the numerator of the equation $p(w_i | w_j) = \frac{e^{\mathbf{v}_{w_j}^o \cdot \mathbf{v}_{w_i}^i}}{\sum_{k=1}^M e^{\mathbf{v}_{w_k}^o \cdot \mathbf{v}_{w_i}^i}}.$
 - this, in turn implies that the dot product with vectors that are not in the context of w_i will be minimized

- however, the number of context words in word2vec is equal to the size of the vocabulary, M , which may be in the millions or billions.

- the denominator in the equation $p(w_i | w_j) = \frac{e^{\mathbf{v}_{w_j}^o \cdot \mathbf{v}_{w_i}^i}}{\sum_{k=1}^M e^{\mathbf{v}_{w_k}^o \cdot \mathbf{v}_{w_i}^i}}$ is prohibitively expensive to calculate.

- the actual word2vec algorithm thus implements an approximated form:

$$C = - \sum_{i=1}^M \left(\sum_{w_j \in \text{ctx}(w_i)} \log(\sigma(\mathbf{v}_{w_j}^o \cdot \mathbf{v}_{w_i}^i)) + \sum_{w_j \notin \text{ctx}(w_i)} \log(\sigma(-\mathbf{v}_{w_j}^o \cdot \mathbf{v}_{w_i}^i)) \right)$$

- where the first sigmoid maximizes the proximity of input vectors with the output vectors of words in context, whereas the second sigmoid minimizes the proximity of input vectors to output vectors of words not in context, due to the negative sign in the sigmoid parameter
- no conditioned probability (computationally harder to normalize); also, the second term employs a small sample of words that do not occur in the context of w_i

the classification task

- skip-gram trains a probabilistic classifier that, given a test target word w and its context window of L words $c_{1:L}$, assigns a probability based on how similar this context window is to the target word.
 - the probability is based on applying the logistic function to the dot product of the embeddings of the target word with each context word.

embeddings learning

learning skip-gram embeddings

- skip-gram learns embeddings by starting with random embedding vectors and then iteratively **shifting the embedding of each word w to be more like the embeddings of words that occur nearby in texts**, and less like the embeddings of words that do not occur nearby

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

... lemon, a [tablespoon of **apricot** jam, a] pinch ...

c1 c2 w c3 c4

- the target is *apricot*, and we have 4 context words in the $L = \pm 2$ window
- to train a binary classifier we also need negative examples: skipgram with negative sampling (SGNS) uses more negative examples than positive examples (with the ratio between them set by a parameter k).
 - for each of these (w, c_{pos}) training instances we create k negative samples, each consisting of the target w plus a 'noise word' c_{neg}
 - a noise word is a random word from the lexicon, constrained not to be the target word w

learning

- the goal of the learning algorithm is to adjust those embeddings to
 - maximize the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive examples
 - minimize the similarity of the (w, c_{neg}) pairs from the negative examples
- that is, we want to maximize the dot product of the word with the actual context words, and minimize the dot products of the word with the k negative sampled non-neighbor words.

learned matrices

- the skip-gram model learns **two separate embeddings** for each word i : **the target embedding w_i** and **the context embedding c_i** , stored in two matrices, the target matrix $\mathbf{W}$ and the context matrix $\mathbf{C}$.
 - these are **typically added together**, representing word i with the vector $\mathbf{w}_i + \mathbf{c}_i$.
 - **alternatively the $\mathbf{C}$ matrix is thrown away** and each word i is represented by the vector w_i .

final remarks

- the vectors learned by word2vec have been shown to capture semantic information that has a similar impact on downstream applications as the vectors learned through the more expensive low-rank approximation strategy
- this semantic information can also be directly analyzed.
 - for example, the relation between countries and their capital cities (shown as the difference between the two respective vectors) tends to be same regardless of country and capital, that is the distance between the vectors *China* $\rightarrow$ *Beijing* is similar to that between *Portugal* $\rightarrow$ *Lisbon*
- interestingly enough, it has been experimentally observed that algorithms that learn numerical representations amplify the various biases present in the training data, e.g., gender stereotypes.
 - for example it has been observed that the difference between vectors for *father* and *doctor* is similar to that intervening between *mother* and *nurse*.

w2v: technical details

w2v downloads

- GoogleNews-vectors-negative300.bin.gz models and vectors can be downloaded from

<https://drive.google.com/file/d/0B7XkCwpl5KDYNINUTTlSS21pQmM/edit>

- 1.62GB download, and 3.65 GB storage required

details to train your own W2V models

- to train a word2vec model, we first extract the corpus vocabulary and generate from it a random set of initial word vectors.
 - the algorithm then improves their predictive power by changing their weights, based on sampling contexts (where the word exists) and negative contexts (where the word doesn't exist).
- the most relevant parameters employed by the genesis Word2Vec class are surveyed in the following.

Preparing Data for Word2Vec

- before training a Word2Vec model, we need to preprocess our text data.
 - this involves tokenization, lowercasing, and removing punctuation. we'll use the NLTK library for these tasks.

object definition and default settings

- next slides provide a description
`gensim.models.Word2Vec` interface

	parameter	default	description
Input data and corpus	sentences	None	Iterable of tokenized sentences used for training. If None, no in-memory corpus is supplied.
	corpus_file	None	Path to a text corpus on disk. Alternative to sentences, useful for large datasets.
Vector size and context	vector_size	100	Dimensionality of word embeddings. Each word is represented by a 100-dimensional vector.
	window	5	Maximum context window size. Up to 5 words left and 5 words right are considered context.
	epochs	5	Number of full passes over the training corpus.

Learning-rate settings	min_count	5	Ignores words appearing fewer than 5 times.
	max_vocab_size	None	No temporary limit on vocabulary size during construction.
	max_final_vocab	None	No desired final vocabulary cap.
	sample	0.001	Downsampling threshold for very frequent words. Helps reduce dominance of common words.
	trim_rule	None	No custom rule for keeping/discarding words. Standard pruning applies.
	sorted_vocab	1	Vocabulary is sorted by descending frequency before assigning word indices.

Training architecture	sg	0	Chooses architecture: 0 = CBOW, 1 = Skip-gram. Default is CBOW.
	cbow_mean	1	In CBOW, context vectors are averaged rather than summed.
	shrink_windows	TRUE	Randomly reduces effective window size during training, following original word2vec behavior.
Output-layer strategy	hs	0	Hierarchical softmax is disabled.
	negative	5	Uses negative sampling with 5 negative examples per target word.
	ns_exponent	0.75	Shapes the negative-sampling distribution; 0.75 is the classic word2vec choice.

Randomness, speed, and batching	seed	1	Random seed for initialization and reproducibility.
	workers	3	Number of parallel worker threads used in training.
	hashfxn	hash	Python built-in hash function used internally.
	batch_words	MAX_WORDS_IN_BATCH	Target batch size for training jobs; in practice typically about 10,000 words.
Advanced / miscellaneous options	null_word	0	No special null/padding word is added.
	compute_loss	FALSE	Training loss is not computed, which saves time. The model still does gradient-based training with the selected architecture.
	callbacks	()	No callback functions are used during training.
	comment	None	No extra metadata/comment attached to the model.

further static embeddings

pre-trained word embeddings

- various algorithms exist to train word embeddings, such as word2vec, GloVe (from the Stanford University) and fastText (from Facebook research)
 - in all cases both software to train the embeddings and the pre-trained embeddings are provided
 - most open-domain word embeddings are trained on large corpora such as Wikipedia and Gigaword
 - pre-trained embeddings are usually distributed as a text file in which each line represents a word vector, such as

house 0.60137 0.28521 -0.032038 -0.43026 0.74806 0.26223 ...

fasttext

- fasttext is an extension of word2vec
 - word2vec has no good way to deal with **unknown words** (that appear in a test corpus but were unseen in the training corpus)
 - a related problem is **word sparsity** (where some of the many forms for each noun and verb may only occur rarely)
- fasttext deals with these problems by using **subword models**, representing each word as itself plus a bag of constituent n-grams, with special boundary symbols < and > added to each word.
 - for example, with $n = 3$ the word 'where' would be represented by the sequence <where> plus the character n-grams: **<wh, whe, her, ere, re>**

fasttext

- then a skipgram embedding is learned for each constituent n-gram, and the word 'where' is represented by the sum of all of the embeddings of its constituent n-grams.
 - unknown words can then be presented only by the sum of the constituent n-grams.
 - the fasttext open-source library, including pretrained embeddings for 157 languages is available at <https://fasttext.cc>

GloVe: Global Vectors for Word Representation

- we use the `glove.6B.300d.txt` embeddings, available at the URL <https://nlp.stanford.edu/projects/glove/>
 - 400 k word embeddings are provided herein, arranged into 300 dimensions, trained on a Wikipedia 2014 dump and on Gigaword 5
- we explore these embeddings through the `gensim` Python library

GloVe: Global Vectors for Word Representation

- loading embeddings and assigning them to the `glove` variable is done in few lines of code

```
from gensim.models import KeyedVectors
```

```
fname = "path/glove.6B.300d.txt"
```

```
glove = KeyedVectors.load_word2vec_format(fname, no_header=True)
```

```
glove.vectors.shape
```

```
(400000, 300)
```

GloVe: Global Vectors for Word Representation

- the class `KeyedVectors` provides a method called `most_similar` that receives a word and computes its cosine similarity to all other embeddings, and returns the `topn` most-similar words. By default, `topn` is set to 10.

```
glove.most_similar("cactus")
```

```
[('cacti', 0.6634564399719238),  
 ('saguaro', 0.6195855140686035),  
 ('pear', 0.5233486890792847),  
 ('cactuses', 0.5178282260894775),  
 ...  
 ('peyote', 0.45344963669776917),  
 ('succulents', 0.45127877593040466)]
```

Daniele Radicioni - TLN

analogy

- the semantics grasped by word embeddings goes beyond word similarity
 - for example, a well-known analogy that highlights gender information is observed by handling embedding vectors: *king* – *man* $\approx$ *queen* – *woman*, that amounts to saying: "man is to king what woman is to queen".
 - one can subtract the meaning of *man* and add the meaning of *woman* to obtain the definition of female royalty: *queen* $\approx$ *king* – *man* + *woman*.

analogy

- one can subtract the meaning of *man* and add the meaning of *woman* to obtain the definition of female royalty:
 - $queen \approx king - man + woman$.

```
# king - man + woman  
glove.most_similar(positive=["king", "woman"], negative=["man"])
```

```
[ ('queen', 0.6713277101516724),  
  ('princess', 0.5432624220848083),  
  ('throne', 0.5386105179786682),  
  ...  
  ('prince', 0.4668240249156952),  
  ('wife', 0.46473270654678345) ]
```

analogy

- what do we expect from this?
 - *berlin - germany + australia $\approx$???*

```
# berlin - germany + australia  
glove.most_similar(positive=["berlin", "australia"], negative=["germany"])
```

```
[('sydney', 0.6780862212181091),  
 ('melbourne', 0.6499180793762207),  
 ('australian', 0.594883143901825),  
 ('perth', 0.5828552842140198),  
 ('canberra', 0.5610732436180115),  
 ...]
```

implementation details

- embedding vectors are stored in the attribute `vector` from the object `KeyedVectors`
 - vector is a 2-dimensional NumPy array, where each row corresponds to a word in the vocabulary
 - but normalized embeddings can be obtained using the `get_normed_vectors` method.

```
normed_vectors = glove.get_normed_vectors()  
normed_vectors.shape
```

implementation details

- the `KeyedVectors` object stores this mapping between words and the matrix rows in a list of terms called `index_to_key`, and a term-to-index dictionary called `key_to_index`.

```
word_list = glove.index_to_key  
print(word_list[:10])
```

```
#glove.key_to_index
```

```
dict_words_2_indices = glove.key_to_index  
print(list(dict_words_2_indices.items())[ :10])
```

LAB: IR and text categorization with VSM

- These assignments include different parts: each will be provided with its own score, so that everyone is allowed to choose what best suits her/his preferences.

Document Similarity and Retrieval using the VSM

Document Similarity and Retrieval using the VSM

A. With Sparse Embeddings

- Use the News Category Dataset available on Kaggle:
<https://www.kaggle.com/datasets/rmisra/news-category-dataset>
- This dataset contains ~200k news headlines from HuffPost, classified into categories such as politics, tech, and sports.
- Setup and Preprocessing
 - Load the dataset and randomly select a subset of headline texts (e.g., 10,000).
 - Preprocess the text through tokenization, lowercasing, stopwords and punctuation removal, lemmatization.

Document Similarity and Retrieval using the VSM

- Vector Space Model representation
 - Convert the preprocessed documents to TF-IDF vectors (use `TfidfVectorizer` from `scikit-learn`)
 - Store the resulting matrix and vocabulary
- Document Retrieval
 - Prompt the user for a query (e.g., "government budget")
 - Preprocess the query using the same pipeline
 - Convert the query to a TF-IDF vector using the same vectorizer
 - Compute the cosine similarity between the query and all documents
 - Return and display the top 5 most relevant documents

Document Similarity and Retrieval using the VSM

- Analysis
 - Manually inspect the top results: do they seem relevant?
 - Try at least 10 queries from different categories and observe the results

Document Similarity and Retrieval using the VSM

B. With the Rocchio Method

- Data selection
- There are a total of 42 news categories in the dataset. The top-15 categories and corresponding article counts cover above 70% of the whole data, and are as follows
 - Select only news from these classes, and apply a 90/10 split (90% for building profiles, and 10% for testing).

- POLITICS: 35602
- WELLNESS: 17945
- ENTERTAINMENT: 17362
- TRAVEL: 9900
- STYLE & BEAUTY: 9814
- PARENTING: 8791
- HEALTHY LIVING: 6694
- QUEER VOICES: 6347
- FOOD & DRINK: 6340
- BUSINESS: 5992
- COMEDY: 5400
- SPORTS: 5077
- BLACK VOICES: 4583
- HOME & LIVING: 4320
- PARENTS: 3955

Document Similarity and Retrieval using the VSM

- Categorization task
 - The task is to categorize the items in the test-set.
 - Accuracy over classes must be recorded, as well as the average over the 15 classes.
 - split data into 10 folds (90/10), repeat the classification as many times, and record average accuracy.

Document Similarity and Retrieval using the VSM

- The solution should be based on the following steps:
 - Preprocessing
 - Clean and normalize text (lowercase, punctuation removal, etc.), tokenize and remove stopwords
 - Use `TfidfVectorizer` from scikit-learn to convert documents to TF-IDF vectors
 - Split data into training and test sets
 - For each class compute the centroid based on the set of training documents for that class
 - For each test document compute cosine similarity to all class centroids
 - Assign the label of the closest centroid

Document Similarity and Retrieval using the VSM

C. With Dense Embeddings

- In this assignment we use dense (Word2Vec) embeddings within the Vector Space Model (VSM) framework.
- We start by training a model, whose vectors will then be employed to represent the documents
 - create a text corpus using the aforementioned News Category Dataset
 - preprocess the text through tokenization, lowercasing, and stopwords and punctuation removal.
 - train a Word2Vec Model (using `gensim.models.Word2Vec`):
 - choose hyperparameters: `vector_size`, `window`, `min_count`, `sg`
 - save the trained model for later use

Document Similarity and Retrieval using the VSM

- The Vector Space Model is a general framework where text items (e.g., words, documents, sentences) are represented as vectors in an n -dimensional space, and similarity is computed using vector distances (typically cosine similarity).
- While VSM is sparse and based on word frequency, Word2Vec produces dense and semantic word vectors. These can be used to represent not just words but also larger units like sentences and documents by aggregation.

Document Similarity and Retrieval using the VSM

- different from the previous version (Part A), we now represent a document as a vector as follows:
 - Average the Word2Vec vectors of all the words in the document (after preprocessing).
 - Use a weighted average, weighting each word vector by its TF-IDF score.
 - Compare the results obtained on the same set of queries employed in Part A.

Document Similarity and Retrieval using the VSM

D. implement linear discriminant analysis with Dense Embeddings

- in this assignment we also use dense (Word2Vec) embeddings within the Vector Space Model (VSM) framework.
- we represent each document by the mean of its word embeddings, obtaining a dense vector in $\mathbb{R}^d$.

data

- Use this toy dataset composed of 20 short messages, balanced across the two classes.
- Class 1: spam
 1. win a free prize now
 2. claim your free reward today
 3. exclusive offer just for you
 4. click now to win money
 5. free vacation waiting for you
 6. urgent offer claim your prize
 7. limited offer free bonus now
 8. congratulations you win cash
 9. click here for free gift
 10. get your reward immediately
- Class 0: not spam
 11. project meeting scheduled for tomorrow
 12. please review the attached report
 13. the meeting room is available
 14. lunch with the team tomorrow
 15. please send the updated document
 16. the project deadline is next week
 17. can we reschedule the meeting
 18. the report is ready for review
 19. team discussion about the project
 20. please confirm tomorrow appointment

splits

- a simple split is 80/20, e.g.,
 - training set: first 8 spam + first 8 non-spam
 - test set: remaining 2 spam + remaining 2 non-spam
- record classification accuracy in a 5-folds experiment
- you can use libraries for preprocessing and vectorization, but the LDA classifier itself must be implemented by yourselves.

hints for implementation

- to force a dense VSM, **define a small fixed vocabulary** such as:
 $V = (\text{free, offer, win, prize, click, meeting, project, report, team, tomorrow})$
- this gives a document vector in $\mathbb{R}^{10}$.
- each message is then represented as $x = (x_1, \dots, x_{10})^T$ where each component can be:
 - binary presence/absence
 - term frequency
 - TF-IDF weight

hints for implementation

- for each class $k \in \{1, \dots, K\}$, estimate:

- the prior $\hat{\pi}_k = \frac{n_k}{n}$

- the class mean vector $\hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i$

- then estimate the pooled covariance matrix

$$\hat{\Sigma} = \frac{1}{n - K} \sum_{k=1}^K \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

- finally compute, for each class: $\delta_k(x) = x^T \hat{\Sigma}^{-1} \hat{\mu}_k - \frac{1}{2} \hat{\mu}_k^T \hat{\Sigma}^{-1} \hat{\mu}_k + \log \hat{\pi}_k$

- and assign the document to the class with largest score, $\hat{y}(x) = \arg \max_k \delta_k(x)$

Word Clustering Based on Word2Vec Representation

Clustering based on W2V

- Train a Word2Vec Model
 - create a text corpus (e.g., Wikipedia dump, or works from the Project Gutenberg, such as <https://www.gutenberg.org/ebooks/>).
 - preprocess the text through tokenization, lowercasing, and stopwords and punctuation removal.

Clustering based on W2V

- train a Word2Vec Model (using `gensim.models.Word2Vec`):
 - choose hyperparameters: `vector_size`, `window`, `min_count`, `sg`
 - save the trained model for later use
- explore the Model:
 - use `.most_similar()` to explore semantic neighborhoods
 - visualize some vectors using PCA or t-SNE

Clustering based on W2V

- Words Clustering
 - Choose a list of words
 - Cluster them using scikit-learn's KMeans
 - Visualize the resulting clusters using t-SNE or PCA

